

SESSION-6:

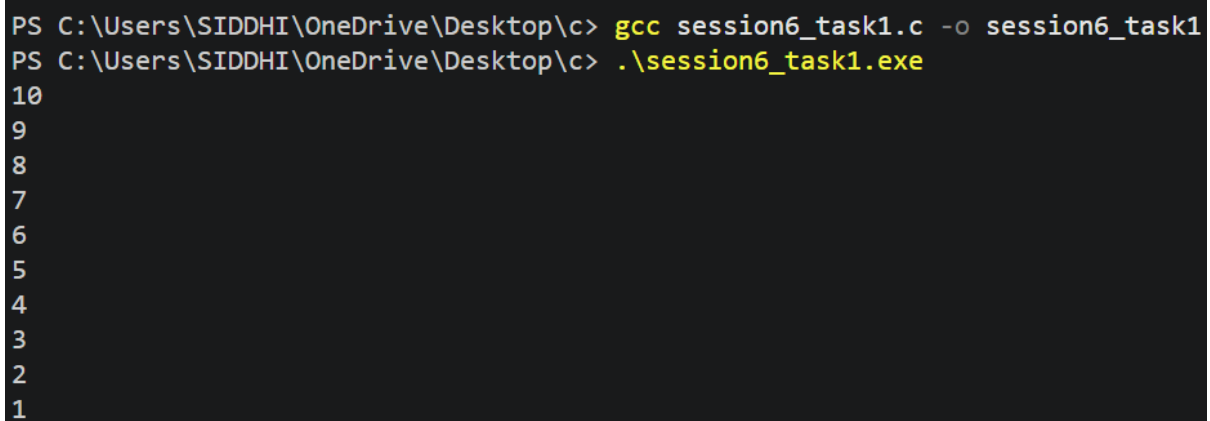
Tasks

1.

Write a program that prints the numbers from 10 down to 1 using a for loop, similar to a countdown timer.

```
#include <stdio.h>
```

```
int main() {  
    for (int i = 10; i >= 1; i--) {  
        printf("%d\n", i);  
    }  
  
    return 0;  
}
```



```
PS C:\Users\SIDDHI\OneDrive\Desktop\c> gcc session6_task1.c -o session6_task1  
PS C:\Users\SIDDHI\OneDrive\Desktop\c> .\session6_task1.exe  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1
```

2.

Create a menu-driven console app that lets the user: 1) View your favorite 3 IPL teams, 2) Add a new team, 3) Exit. Use a while loop to keep showing the menu until the user chooses Exit.

Hint: Use input() (or Scanner in Java) to get the user's choice each time. const readline = require("readline");

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char teams[10][50] = {
```

```
        "Mumbai Indians",
```

```
        "Chennai Super Kings",
```

```
        "Royal Challengers Bengaluru"
```

```
    };
```

```
    int teamCount = 3;
```

```
    int choice;
```

```
    while (1) {
```

```
        printf("\n1. View Favorite IPL Teams\n");
```

```
        printf("2. Add a New Team\n");
```

```
        printf("3. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        getchar();
```

```
        if (choice == 1) {
```

```
            printf("\nFavorite IPL Teams:\n");
```

```
            for (int i = 0; i < teamCount; i++) {
```

```
                printf("%d. %s\n", i + 1, teams[i]);
```

```
            }
```

```
        }
```

```
else if (choice == 2) {
    if (teamCount < 10) {
        printf("Enter new IPL team: ");
        fgets(teams[teamCount], 50, stdin);
        teams[teamCount][strcspn(teams[teamCount], "\n")] = '\0';
        teamCount++;
        printf("Team added successfully!\n");
    }
    else {
        printf("Team list is full!\n");
    }
}
else if (choice == 3) {
    printf("Exiting...\n");
    break;
}
else {
    printf("Invalid choice!\n");
}
}

return 0;
```

```

}
PS C:\Users\SIDDHI\OneDrive\Desktop\c> gcc session6_task2.c -o session6_task2
PS C:\Users\SIDDHI\OneDrive\Desktop\c> .\session6_task2.exe

1. View Favorite IPL Teams
2. Add a New Team
3. Exit
Enter your choice: 1

Favorite IPL Teams:
1. Mumbai Indians
2. Chennai Super Kings
3. Royal Challengers Bengaluru

1. View Favorite IPL Teams
2. Add a New Team
3. Exit
Enter your choice: 3
Exiting...

```

3.

Build a 'Guess the Song' game like Spotify — the program randomly picks a song name from a list and asks the user to guess it. Use a do-while loop so the user can keep guessing until they get it right.

Constraint: Use at least 3 song names of your choice.

```

#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <time.h>

```

```

int main() {

    char songs[3][50] = {

        "Perfect",

        "Believer",

        "Shape of You"

    };
}

```

```
char guess[100];
int selectedSong;

srand(time(NULL));
selectedSong = rand() % 3;

do {
    printf("Guess the song: ");
    fgets(guess, sizeof(guess), stdin);
    guess[strcspn(guess, "\n")] = '\0';

    if (strcmp(guess, songs[selectedSong]) == 0) {
        printf("Correct! You guessed the song!\n");
    } else {
        printf("Wrong guess! Try again.\n");
    }
} while (strcmp(guess, songs[selectedSong]) != 0);

return 0;
```

```

}
PS C:\Users\SIDDHI\OneDrive\Desktop\c> gcc session6_task3.c -o session6_task3
PS C:\Users\SIDDHI\OneDrive\Desktop\c> .\session6_task3.exe
Guess the song: belieever
Wrong guess! Try again.
Guess the song: shape of you
Wrong guess! Try again.
Guess the song: believer
Wrong guess! Try again.
Guess the song: perfect
Wrong guess! Try again.
Guess the song: Perfect
Wrong guess! Try again.
Guess the song: Believer
Wrong guess! Try again.
Guess the song: Shape of You
Correct! You guessed the song!

```

4.

Explain with your own example the difference between entry-controlled and exit-controlled loops by writing a short code snippet for each (for/while vs do-while) and describing what happens if the loop condition is false at the start.

1. Entry-Controlled Loop

A while loop is an entry-controlled loop.

The condition is checked before the loop runs.

```
#include <stdio.h>
```

```

int main() {
    int count = 1;

    while (count <= 3) {
        printf("%d\n", count);
        count++;
    }

    return 0;
}

```

False at the start: If the condition is false initially, the loop executes 0 times.

2. Exit-Controlled Loop

A do-while loop is an exit-controlled loop.

The condition is checked after the loop runs.

```
#include <stdio.h>
```

```
int main() {  
    int count = 5;  
  
    do {  
        printf("%d\n", count);  
        count++;  
    } while (count < 3);  
  
    return 0;  
}
```

False at the start: Even if the condition is false initially, the loop executes at least once.